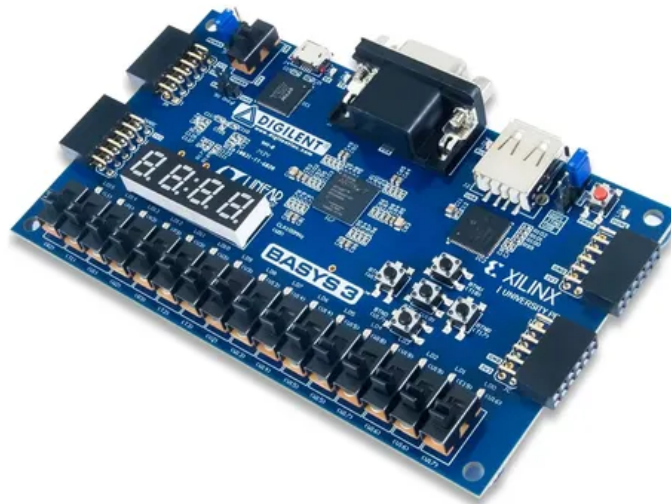


Trabajo Final — Procesador RISC-V - 5 stages

Implementación sobre FPGA Basys-3 (Artix-7 xc7a35tcpg236-1)



Alumno: Alex Agustin, Hernando

Matricula: 42440971

Fecha: 14/07/2026

Cátedra: Arquitectura de Computadoras



Trabajo Final — Procesador RISC-V - 5 stages	1
Introducción	1
Objetivo	2
Diagrama completo	3
Etapas del Pipeline	4
IF (Instruction Fetch)	4
ID (Instruction Decode)	4
EX (Execute)	5
MEM (Memory Access)	5
WB (Write Back)	5
Buffers de pipeline (latches intermedios)	5
Unidad de detección de riesgos (Hazard Detection Unit)	6
Unidad de cortocircuito (Forwarding Unit)	7
Riesgos de control — Branch y Jump	7
Debug Unit	8
7.2 Máquina de estados	9
El gating del pipeline (clock no intervenido)	10
UART, ensamblador y GUI	10
UART	10
Ensamblador y GUI (lado PC)	11
Modos de operación y HALT	11
HALT	11
Modos	12
Demo	12
Análisis del Pipeline	12
Análisis de tiempos	13
Metodología	14
Conclusión	16
Bibliografía	17

Introducción

RISC-V es una arquitectura de conjunto de instrucciones (ISA) abierta y modular, de tipo **RISC** (conjunto reducido y regular de instrucciones). Este trabajo implementa el subconjunto entero de 32 bits (**RV32I**) sobre un **procesador segmentado de cinco etapas** (*Patterson & Hennessy: Computer Organization and Design: RISC-V Edition*).

Características principales del diseño:

- **Segmentación (pipelining):** la ejecución se divide en cinco etapas que operan en paralelo sobre instrucciones distintas:
 - **IF** — *Instruction Fetch*: búsqueda de la instrucción en la memoria de programa.
 - **ID** — *Instruction Decode*: decodificación y lectura del banco de registros.
 - **EX** — *Execute*: operación aritmético-lógica y cálculo de destinos de salto.
 - **MEM** — *Memory Access*: lectura/escritura de la memoria de datos.
 - **WB** — *Write Back*: escritura del resultado en el banco de registros.
- **Banco de registros fijo:**
 - 32 registros de propósito general ($x0-x31$).
 - $x0$ cableado a cero.
 - El ancho del datapath es 32 bits.
 - PC de 64 bits.
- **Modelo *load/store*:** las operaciones aritmético-lógicas se hacen entre registros; la transferencia con memoria es exclusiva de *lw/sw* (y sus variantes de *byte/half*).
- **Tres formatos base de instrucción** (más U/B/J para inmediatos):
 - **Tipo R** — operaciones registro-registro.
 - **Tipo I** — inmediatos, loads y jalr.
 - **Tipo S/B** — stores y branches.
 - **Tipo U/J** — lui y jal.

El procesador se controla desde una PC mediante **UART**: se carga el programa, se ejecuta (en modo continuo o paso a paso) y se vuelca el estado completo del procesador de vuelta a la PC.

Objetivo

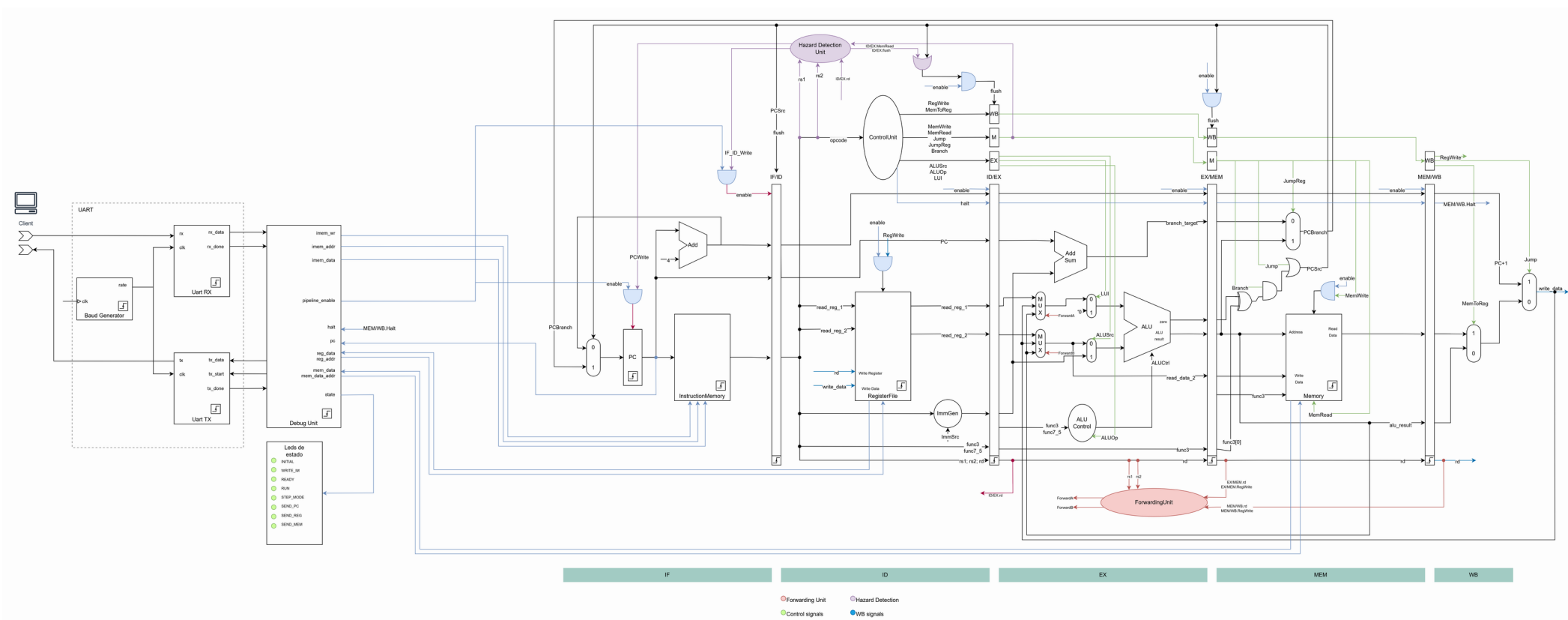
Implementar un procesador RISC-V segmentado en las etapas IF, ID, EX, MEM y WB, capaz de ejecutar el siguiente subconjunto de instrucciones:

Grupo	Instrucciones RISC-V implementadas	Opcode
R-type	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	0110011
I-aritm.	ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI	0010011
Load	LB, LBU, LH, LHU, LW, LWU	0000011
Store	SB, SH, SW	0100011
Branch	BEQ, BNE	1100011
U-type	LUI	0110111
Jump	JAL	1101111
Jump-reg	JALR	1100111
HALT	opcode custom-0 (0x0000000B)	0001011

Requisitos funcionales:

- El procesador debe programarse y reprogramarse por UART.
- El clock no debe ser alterado en ninguna parte de la lógica funcional.
- Una Debug Unit debe enviar: el PC, los 32 registros, el contenido de los latches intermedios y la memoria de datos.
- Dos modos de operación: continuo y paso a paso, con el pipeline vacío al terminar.
- Un ensamblador (Assembly a código máquina) y validación en placa real.
- Análisis de timing y elección de la frecuencia de funcionamiento.

Diagrama completo



Etapas del Pipeline

El núcleo del procesador es el módulo RISC-V, que instancia las cinco etapas y los cuatro buffers de pipeline.

IF (Instruction Fetch)

Módulo InstructionFetch. Busca la próxima instrucción y calcula la dirección siguiente.

- **PC byte-addressed:** el PC cuenta **bytes** e incrementa de a **4** (alineado con Patterson & Hennessy). La memoria de instrucciones se indexa con $PC \gg 2$. Esta decisión se documenta en CONSIDERACIONES.md C-001.
- **Memoria de instrucciones:** arreglo de 2^{NB_ADDR} words de 32 bits, con un puerto de escritura para la carga por la Debug Unit.
- **Selección de PC:** un mux elige entre $PC+4$ y el destino de salto (PCBranch), según PCSrc (resuelto en MEM).

Señal	Dir	Significado
i_enable	in	habilita el PC (congelable por stall / Debug Unit)
i_PCSrc / i_PCBbranch	in	redirección de PC (branch/jump tomado)
i_imem_wr/addr/data	in	escritura de la IM (carga del programa)
o_PC, o_PC_increment	out	PC actual y $PC+4$
o_instruction	out	instrucción buscada

ID (Instruction Decode)

Módulo instructionDecode. Decodifica la instrucción, lee el banco de registros y genera las señales de control. Submódulos:

- **RegisterFile** — 32 registros; x0 cableado a 0; write-through combinacional: la escritura ocurre en el flanco de subida, pero un bypass combinacional en los puertos de lectura adelanta el dato que WB está escribiendo cuando coincide con el registro leído en ID. Logra el mismo efecto que la técnica “escribir en la 1.^a mitad y leer en la 2.^a” (P&H) sin depender de los semiciclos, y resuelve el hazard entre WB e ID. Expone un puerto de lectura de debug.
- **ControlUnit** — decodifica el opcode y emite las señales de control (RegWrite, ALUSrc, ALUOp, MemRead, MemWrite, MemToReg, Branch, Jump, JumpReg, Lui, Halt).
- **ImmediateExtend** — arma y extiende en signo los inmediatos I/S/B/U/J.



EX (Execute)

Módulo ExecuteStage. Ejecuta la operación aritmético-lógica y calcula el destino de los saltos.

- **ALU** — operaciones ADD/SUB/AND/OR/XOR/SLL/SRL/SRA/SLT/SLTU (selección por ALUControl a partir de ALUOp + funct3 + funct7[5]).
- **Muxes de forwarding** en ambos operandos: eligen entre el dato del banco, el resultado de EX/MEM o el dato de MEM/WB.
- **Mux ALUSrc**: elige entre el segundo registro y el inmediato.
- **branch_target = PC + immediate** (directo, sin shift: el inmediato ya es byte offset).

MEM (Memory Access)

Módulo MemoryAccessStage. Acceso a la memoria de datos y **resolución de saltos**.

- **DataMemory** — soporta accesos de **byte** / halfword / word, con carga con signo (lb/lh) y sin signo (lbu/lhu), según funct3. Es word-addressed. Expone un puerto de lectura de debug.
- **Branch/Jump**: evalúa la condición (zero/funct3[0] para beq/bne) y genera PCSrc y PCBranch. La resolución ocurre en el límite EX/MEM (penalidad de 3 ciclos por flush).

WB (Write Back)

Módulo WriteBackStage. Selecciona el dato a escribir en el banco de registros mediante muxes explícitos:

- MemToReg elige entre el resultado de la ALU y el dato leído de memoria.
- Jump selecciona PC+4 como dato a escribir (link de jal/jalr).
- Emite write_data, write_reg y RegWrite de vuelta a la etapa ID.

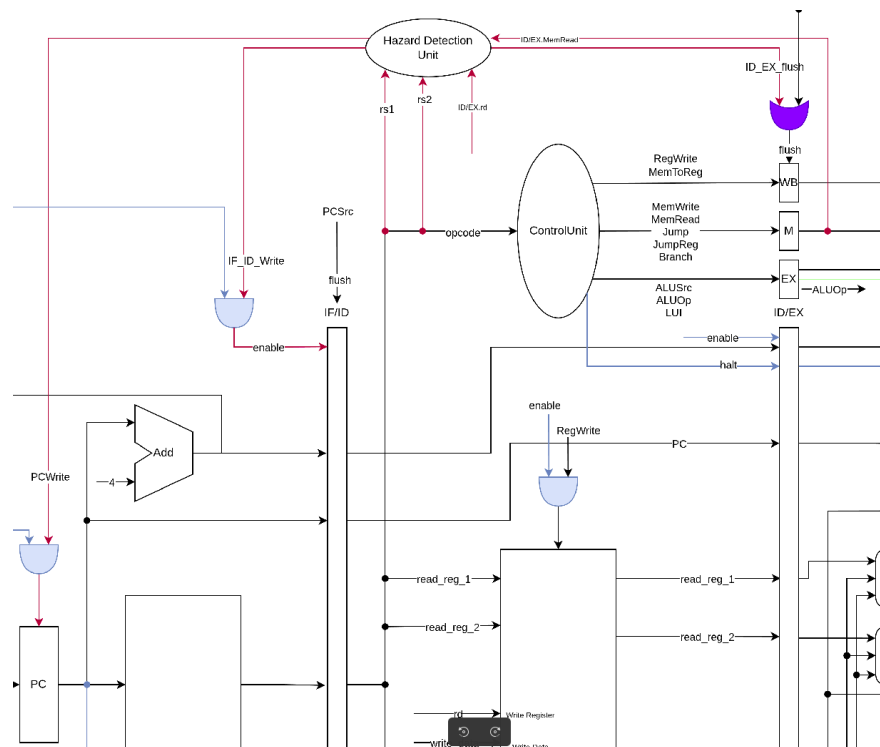
Buffers de pipeline (latches intermedios)

Entre cada par de etapas hay un **buffer registrado** que propaga datos y control:

Buffer	Contenido (resumen)
IF_ID_Buffer	PC, PC+4, instruction
ID_EX_Buffer	PC, PC+4, read_data_1/2, immediate, rs1, rs2, rd, funct3, funct7_5, + 11 señales de control
EX_MEM_Buffer	alu_result, zero, read_data_2, rd, funct3, branch_target, PC+4, + control
MEM_WB_Buffer	alu_result, mem_read_data, rd, PC+4, + control

Estos son precisamente los latches intermedios que la Debug Unit vuelca por UART. Todos comparten el primitivo PosEdgeRegister (flip-flop parametrizado con reset y enable síncronos), y soportan flush (para insertar burbujas / limpiar tras un salto tomado).

Unidad de detección de riesgos (Hazard Detection Unit)

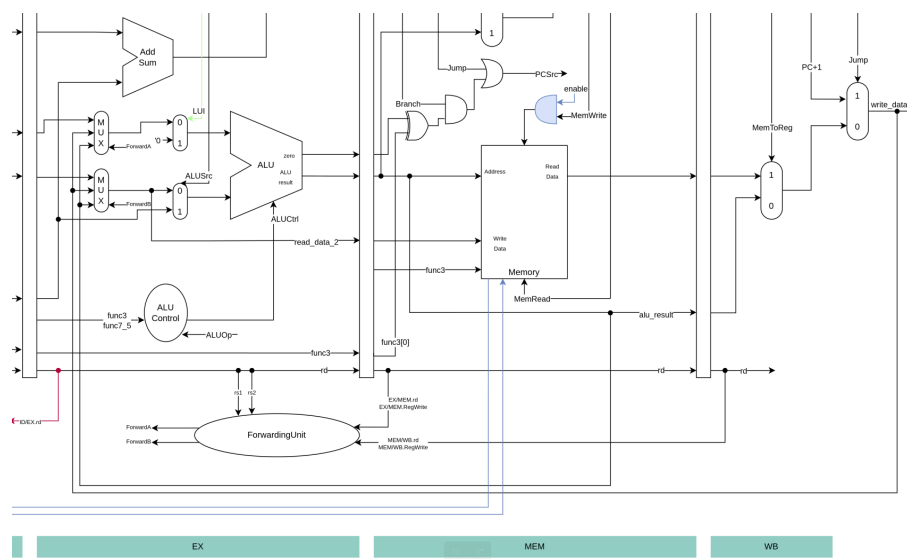


Sección del diagrama completo: Unidad de detección de riesgos, conexión con PC, Buffers IF/ID y ID/EX para el flush

Un riesgo de datos ocurre cuando una instrucción necesita un resultado que todavía no se calculó/escribió. La mayoría se resuelven por *forwarding*, pero el caso *load-use* no: un lw produce el dato al final de MEM, demasiado tarde para adelantarlo a la ALU de la instrucción siguiente.

El stall congela PC e IF/ID (o_PCWrite=0, o_IF_ID_Write=0) e inserta una burbuja poniendo a 0 el control de ID/EX (o_ID_EX_flush=1). Tras un ciclo, el dato ya está en MEM/WB y el forwarding normal lo entrega.

Unidad de cortocircuito (Forwarding Unit)



Sección del diagrama completo: Unidad de detección de cortocircuito, conexión con muxes para seleccionar los operadores de la ALU basándose en registros de los buffers EX/MEM y MEM/WB.

Módulo ForwardingUnit es puramente combinacional y resuelve los riesgos de datos sin perder ciclos, adelantando el resultado desde un buffer posterior directamente a la ALU.

Codificación de los selectores:

Valor	Fuente seleccionada
2'b00	Banco de registros (ID/EX.read_data)
2'b10	EX/MEM (resultado de ALU, distancia 1)
2'b01	MEM/WB (dato de WB: ALU o memoria, distancia 2)

Riesgos de control — Branch y Jump

Los riesgos de control aparecen con branches y saltos: cuando se toma uno, las instrucciones que ya entraron al pipeline detrás de él son incorrectas.

Estrategia: assume-not-taken + flush. Los saltos se resuelven en el límite EX/MEM (inicio de MEM). Cuando PCSrc=1 (salto tomado), se hace flush de IF/ID, ID/EX, EX/MEM (3 ciclos de penalidad) y el PC se redirige a PCBranch.

Debug Unit

La Debug Unit es una máquina de estados que actúa como único puente entre la UART y el pipeline. La UART sólo mueve bytes sueltos; la Debug Unit los interpreta: carga el programa, arranca/frena el core y reporta su estado.

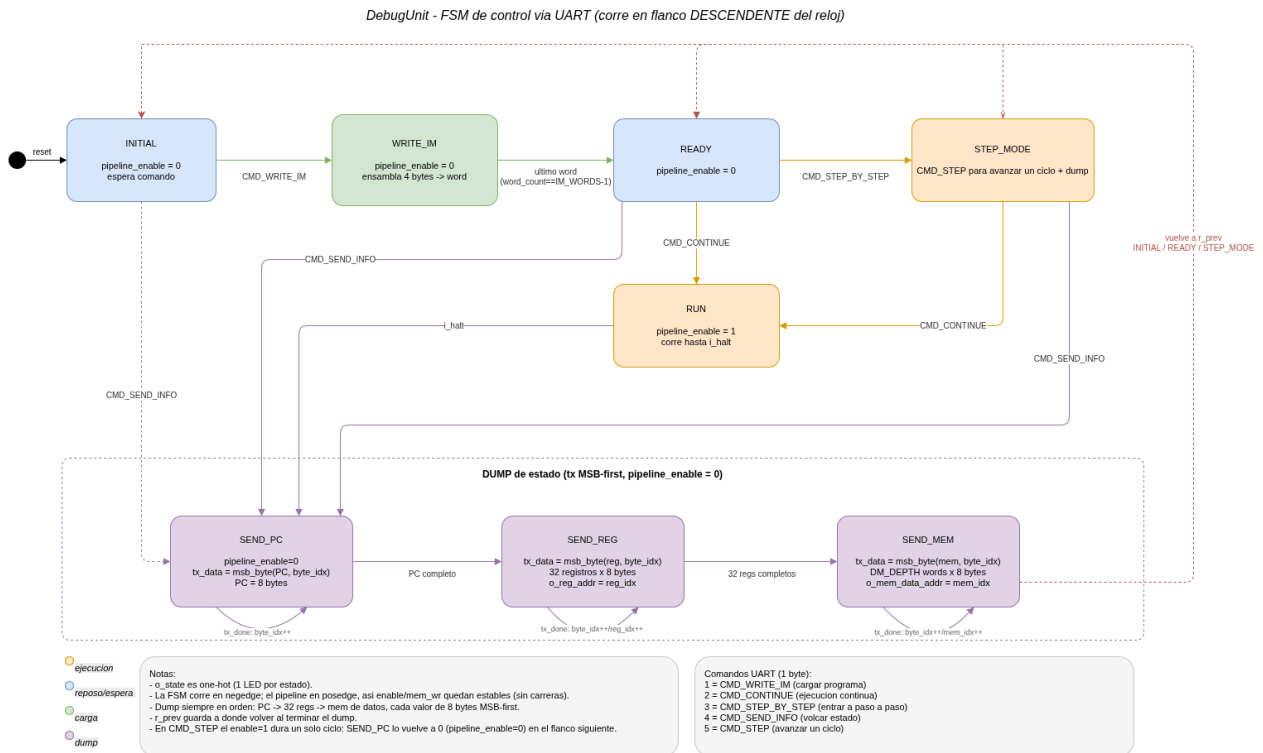
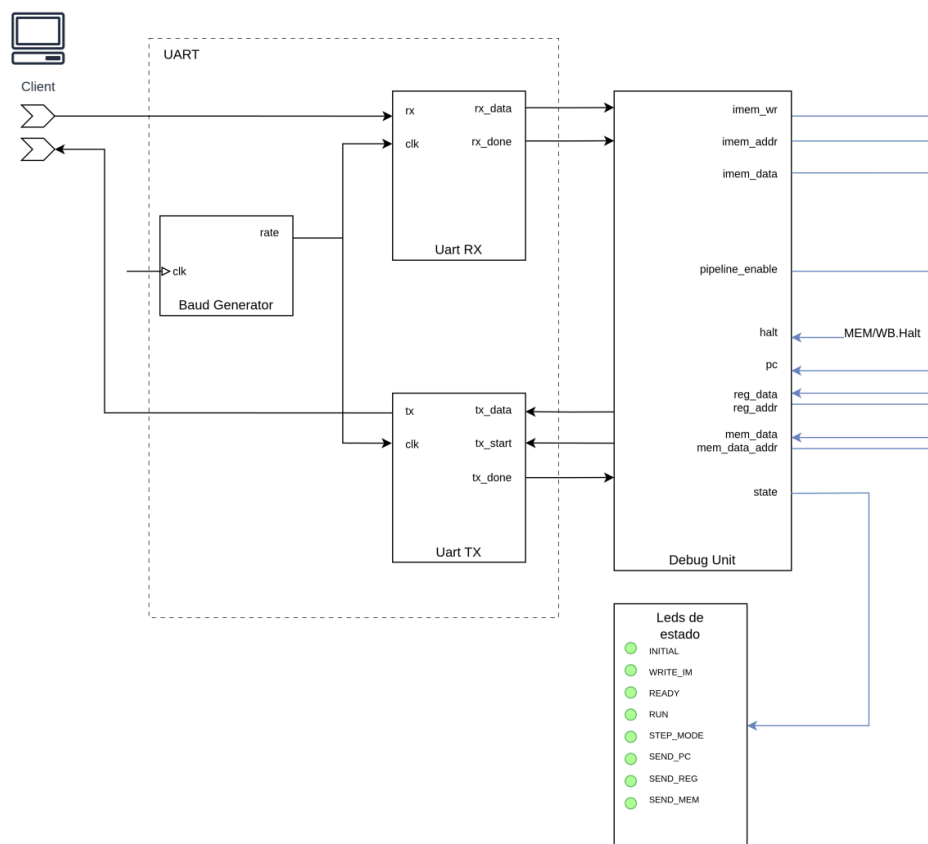


Diagrama de flujo para la máquina de estado de la unidad de debugging, los comandos son recibidos como bits al igual que los datos, todo a través de la UART.

7.2 Máquina de estados



Flujo desde la PC a la unidad de debugging

Para la debug unit, 9 estados, cada uno enciende un LED para depuración visual:

Estado	Descripción
INITIAL	Reposo: espera WRITE_IM o SEND_INFO.
WRITE_IM	Recibe el programa byte a byte y lo escribe en la IM.
READY	Programa cargado: espera CONTINUE / STEP_BY_STEP.
RUN	Ejecución continua hasta HALT.
STEP_MODE	Paso a paso: avanza un ciclo por comando.
SEND_PC	Transmite el PC.
SEND_REG	Transmite los 32 registros.
SEND_MEM	Transmite la memoria de datos.
SEND_LATCH	Transmite los latches intermedios (buffers de pipeline).

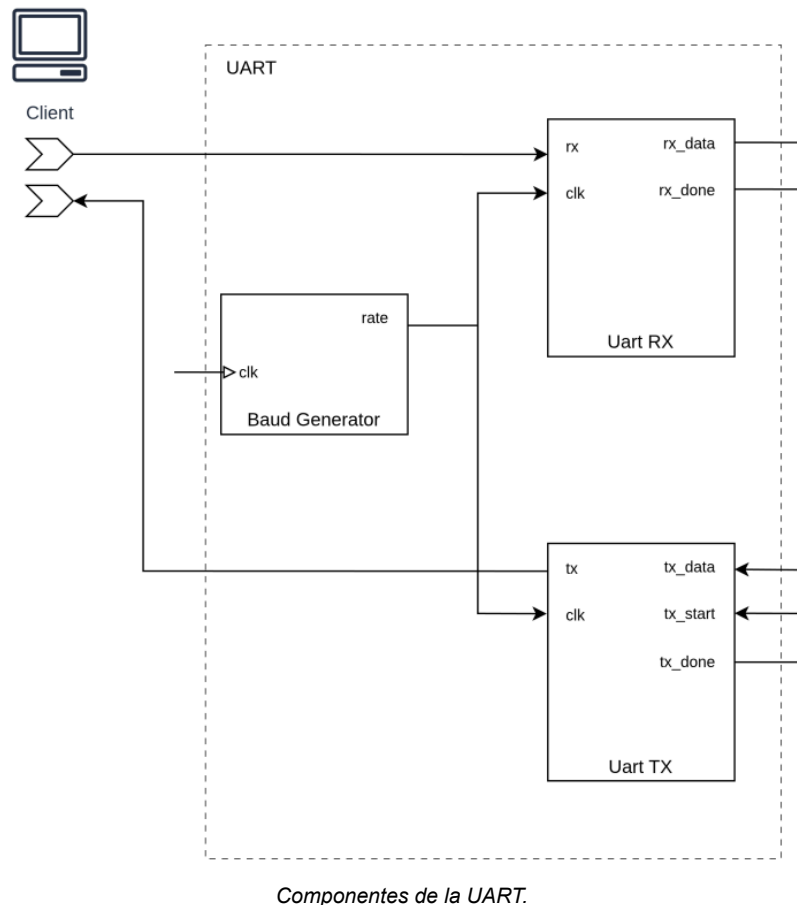
El gating del pipeline (clock no intervenido)

La señal `o_pipeline_enable` entra al core como `i_if_enable` y gatea todo el estado secuencial (PC, los cuatro buffers, escritura al banco y a la memoria de datos). Cuando vale 0, nada avanza, pero el reloj nunca se toca. Esto garantiza que durante la carga y entre pasos el procesador esté quieto y que un pipeline congelado nunca escriba registros ni memoria.

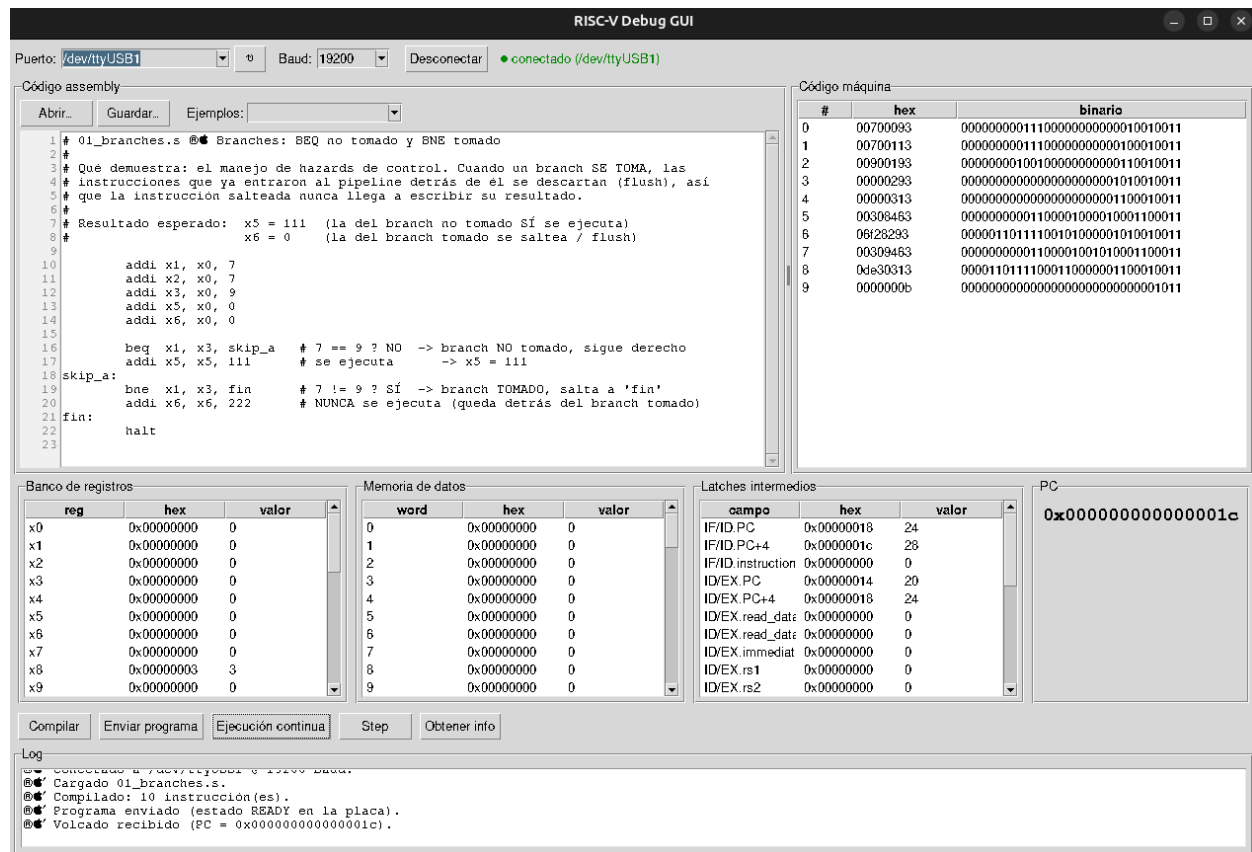
UART, ensamblador y GUI

UART

Implementación: BaudRateGenerator (divisor con sobre-muestreo 16x), UartRx, UartTx y el wrapper full-duplex Uart. Configuración 8N1, 19200 baud.



Ensamblador y GUI (lado PC)



Interfaz gráfica para interactuar con el pipeline.

Programa en Python:

- assembler.py — ensambla Assembly RISC-V (.s/.asm) a código máquina.
- uart.py — capa de protocolo: arma el binario con zero-padding a IM_WORDS, envía comandos y reconstruye el dump (PC + registros + memoria + latches).
- riscv_debug.py — CLI (load, run, step, info, loadrun).
- gui.py — GUI con editor, vista de código máquina y tablas de registros, memoria y latches intermedios.

Modos de operación y HALT

HALT

Opcod HALT dedicado (custom-0 de RISC-V = 0x0000000B). La ControlUnit lo decodifica con todas las demás señales en 0; el bit Halt viaja por los buffers y se señaliza o_halt al llegar a WB, para que la última instrucción real complete su write-back antes de congelar el pipeline.

Se validó el funcionamiento del halt con el siguiente programa:

```
addi x1, x0, 5
addi x2, x0, 5
addi x3, x0, 5
halt
addi x4, x0, 9
```

Al final del programa vemos en los latches intermedios que la última instrucción (después del halt y por lo tanto fuera del programa) pasa por el pipeline hasta llegar al buffer de EX/MEM, en este último ciclo la unidad de debugging congela el pipeline completamente, con la señal MEM/WB.halt.

Modos

- **Continuo:** un comando (0x02) y el core corre hasta HALT; ahí se vuelca todo el estado.
- **Paso a paso:** cada comando (0x05) ejecuta **un ciclo** de reloj y vuelca el estado, permitiendo inspeccionar el avance instrucción a instrucción (incluidos los latches intermedios).

Demo

Análisis del Pipeline

El siguiente análisis muestra cómo las instrucciones del programa avanzan a través de las cinco etapas del pipeline en una arquitectura de procesador que implementa el modelo de ejecución de instrucciones en cinco etapas. El propósito de esta tabla es ilustrar el proceso de ejecución y los posibles conflictos y retrasos que pueden ocurrir en el pipeline, proporcionando una visión clara de cómo cada instrucción es manejada en cada ciclo del reloj.

# 01_branches.s — Branches: BEQ no tomado y BNE tomado		
#		
# Qué demuestra: el manejo de hazards de control. Cuando un branch SE TOMA, las		
# instrucciones que ya entraron al pipeline detrás de él se descartan (flush), así		
# que la instrucción saltada nunca llega a escribir su resultado.		
# tambien podemos ver que x5 y x6 usan la unidad de cortocircuito para obtener x2/x1/x3		
PC		INSTRUCCION
0		addi x1, x0, 7
4		addi x2, x0, 7
8		addi x3, x0, 9
12		addi x5, x2, x1
16		addi x6, x2, x3
20		beq x1, x3, skip_a
24		addi x5, x5, 1
	skip_a:	
28		bne x5, x6, fin
32		addi x6, x6, 1
	fin:	
36		halt

Instrucciones de ejemplo, con saltos condicionales y dependencia de datos

PC	IF	ID	EX	MEM	WB
0	addi x1, x0, 7				
4	addi x2, x0, 7	x1 = 0 + 7			
8	addi x3, x0, 9	x2 = 0 + 7	sumo		
12	addi x5, x2, x1	x3 = 0 + 9	sumo		
16	addi x6, x2, x3	x5 = x2 + x1	sumo		guardo
20	beq x1, x3, skip	x6 = x2 + x3	sumo		guardo
24	addi x5, x5, 1	salto cond	sumo		guardo
28	bne x5, x6, fin	x5 ++	z=0		guardo
32	addi x6, x6, 1	salto cond	sumo	no salto	guardo
36	halt	x6 ++	z=0		
	Flush	Flush	flush	salto	guardo
36	halt	nop	nop	nop	
40		halt	nop	nop	nop
44			halt	nop	nop
48				halt	nop

Ejecución de las instrucciones a través del pipeline de 5 etapas

Análisis de tiempos

Worst Negative Slack (WNS): Cuando el WNS es negativo, significa que el tiempo requerido para el camino combinacional (la ruta por la que viajan los datos) es mayor que el período del reloj. Es decir, el reloj es demasiado rápido para el diseño actual.

Setup Time: Es el tiempo que los datos deben mantenerse estables antes de que el reloj (clock) haga una transición para asegurar que sean capturados correctamente.

Hold Time: Es el tiempo que los datos deben permanecer estables después de la transición del reloj para que el valor sea retenido correctamente.

Metodología

- Configuración del reloj: Se utilizaron diferentes frecuencias de reloj para evaluar el rendimiento del diseño.
- Ejecución de la síntesis y análisis de tiempos: Se utilizaron herramientas de análisis de timing (Vivado) para medir el Worst Negative Slack a cada frecuencia.
- Determinación del path crítico: Identificación del camino combinacional más largo en el diseño, desde la etapa IF hasta la etapa WB del pipeline.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.659 ns	Worst Hold Slack (WHS): 0.139 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 36933	Total Number of Endpoints: 36933	Total Number of Endpoints: 12330

All user specified timing constraints are met.

Design Timing Summary for 65Mhz clk

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0.281 ns	Worst Hold Slack (WHS): 0.032 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): -6.081 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 79	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 36997	Total Number of Endpoints: 36997	Total Number of Endpoints: 12400

Design Timing Summary for 75Mhz clk

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -1.001 ns	Worst Hold Slack (WHS): 0.064 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): -34.055 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 62	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 36921	Total Number of Endpoints: 36921	Total Number of Endpoints: 12326

Timing constraints are not met.

Design Timing Summary for 100Mhz clk

El techo de timing real es ~65 MHz. Por encima de ~70 MHz, la optimización por timing replica registros, explota los control sets y satura el device..

A 75 MHz el setup falla: WNS -0.281 ns, 79 endpoints fallando. Los 10 peores paths van todos desde la DebugUnit (r_mem_wr, r_im_addr) hacia el clock-enable de los registros de InstructionMemory — el camino de carga de programa, con 85 % de net delay (5.5 ns de ruteo vs 1 ns de lógica). Se puede ver en la siguiente tabla los peores paths:

Intra-Clock Paths - clk_out1_clk_wiz_0 - Setup										
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	-0.281	2	3	32	u_debug/ir_mem_wr_reg/C	u_core/IFlu_InstructionMemory/ir_mem_reg[71][6]/CE	6.456	0.963	5.493	6.7
Path 2	-0.250	2	3	32	u_debug/ir_im_addr_reg[2]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[250][1]/CE	6.438	0.702	5.736	6.7
Path 3	-0.245	2	3	32	u_debug/ir_mem_wr_reg/C	u_core/IFlu_InstructionMemory/ir_mem_reg[71][16]/CE	6.428	0.963	5.465	6.7
Path 4	-0.239	2	3	32	u_debug/ir_im_addr_reg[2]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[234][14]/CE	6.425	0.702	5.723	6.7
Path 5	-0.212	2	3	32	u_debug/ir_im_addr_reg[0]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[1][15]/CE	6.610	0.707	5.903	6.7
Path 6	-0.209	2	3	32	u_debug/ir_mem_wr_reg/C	u_core/IFlu_InstructionMemory/ir_mem_reg[55][15]/CE	6.387	0.963	5.424	6.7
Path 7	-0.187	2	3	32	u_debug/ir_im_addr_reg[0]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[139][10]/CE	6.382	0.733	5.649	6.7
Path 8	-0.184	2	3	32	u_debug/ir_im_addr_reg[2]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[249][15]/CE	6.370	0.969	5.401	6.7
Path 9	-0.183	2	3	32	u_debug/ir_im_addr_reg[2]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[255][15]/CE	6.369	0.937	5.432	6.7
Path 10	-0.177	2	3	32	u_debug/ir_im_addr_reg[2]/C	u_core/IFlu_InstructionMemory/ir_mem_reg[249][4]/CE	6.364	0.969	5.395	6.7

Intra Clock Paths

InstructionMemory (y DataMemory) se sintetizan como arrays de flip-flops en fabric, no como BRAM. El culpable es el reset síncrono que limpia todo el array:

Luego de revisar la implementación se observó que la memoria de instrucciones y la memoria de datos no deberían tener un reset síncrono ya que se sintetizan como arrays de flip-flops, no como BRAM.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.634 ns	Worst Hold Slack (WHS): 0.074 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7119	Total Number of Endpoints: 7119	Total Number of Endpoints: 2008

Fixed Design Timing Summary for 75Mhz clk

Luego de quitar los resets en IM y MEM, el total de endpoints cae de 36997 a 7119, las memorias ahora se mapean a BRAM (bloques de la placa dedicados para memoria) real. Finalmente el diseño cierra timing completo a 75 MHz.

Conclusión

Se implementó un procesador RISC-V segmentado de cinco etapas en SystemVerilog, funcional sobre la Basys-3. El pipeline maneja correctamente los tres tipos de riesgos: datos (forwarding + stall load-use), control (flush en branch/jump) y la coherencia lectura/escritura. La interfaz por UART + Debug Unit permite cargar y reprogramar el procesador, ejecutarlo en modo continuo o paso a paso, y volcar el estado completo a una GUI/CLI en Python.

A lo largo de esta implementación se aprendió cómo funciona un procesador a bajo nivel, cuales son las operaciones básicas y cómo funcionan, se tomaron decisiones de diseño como la definición de tamaño y ubicación de registros, donde hacer algunas operaciones y como definir las como código, organización de los archivos y documentación, implementación de la unidad de forwarding e detección de riesgos, y lo que terminó de cerrar el circuito de aprendizaje fue el flujo de comunicación entre la PC y el procesador, es decir, el diseño de la UART y la unidad de debugging, esta máquina de estados es totalmente necesaria para la traducción de los bits (enviados desde la PC) a registros útiles para el procesador. Todo esto formó parte de un gran aprendizaje, realmente útil y que termina de complementar conceptos aprendidos a lo largo de la carrera.

Bibliografía

- D. Patterson, J. Hennessy. *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition*.
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*.
- Digilent. *Basys-3 Reference Manual y Basys-3 Master XDC*.

Repositorio con el código: <https://github.com/agustinhernando2/tp-final-risc-v-pipeline-processor>